

Zadania

Zadanie 1. Rekurencja na liczbach

Proszę zrobić funkcję `sumaKwadratow`, która dla liczby `n` oblicza sumę kwadratów liczb od 1 do `n`. Dla `n <= 0` funkcja powinna zwrócić 0.

```
sumaKwadratow :: Int -> Int
-- przykład: sumaKwadratow 3 == 14, bo 1^2 + 2^2 + 3^2 = 14
```

Zadanie 2. Liczenie elementów spełniających warunek

Proszę zrobić funkcję `policzDodatnie`, która rekurencyjnie zlicza liczby dodatnie na liście liczb całkowitych.

```
policzDodatnie :: [Int] -> Int
-- przykład: policzDodatnie [-2, 0, 5, 7, -1] == 2
```

Zadanie 3. Własna funkcja `all`

Proszę zrobić funkcję `wszystkie`, która sprawdza, czy wszystkie elementy listy spełniają podany predykat. Dla listy pustej wynik powinien być `True`.

```
wszystkie :: (a -> Bool) -> [a] -> Bool
-- przykład: wszystkie even [2,4,6] == True
-- przykład: wszystkie (>0) [1,-3,4] == False
```

Zadanie 4. Usuwanie wartości z listy

Proszę zrobić funkcję `usunWartosc`, która usuwa z listy wszystkie wystąpienia wskazanej wartości.

```
usunWartosc :: Eq a => a -> [a] -> [a]
-- przykład: usunWartosc 3 [1,3,2,3,4] == [1,2,4]
```

Zadanie 5. Praca na krotce

Proszę zrobić funkcję `poprawWynik`, która dla krotki (nazwa, punkty) zwiększa liczbę punktów o podaną wartość. Nazwa powinna pozostać bez zmian.

```
type Wynik = (String, Int)

poprawWynik :: Int -> Wynik -> Wynik
-- przykład: poprawWynik 5 ("Ala", 12) == ("Ala", 17)
```

Zadanie 7. Lista krotek

Proszę zrobić funkcję `sumaPunktow`, która oblicza łączną liczbę punktów z listy wyników typu `Wynik`.

```
sumaPunktow :: [Wynik] -> Int
-- przykład: sumaPunktow [("Ala",10),("Jan",7)] == 17
```

Zadanie 8. Najlepszy wynik

Proszę zrobić funkcję `najlepszyWynik`, która zwraca osobę z największą liczbą punktów. Dla listy pustej funkcja powinna zwrócić `Nothing`.

```
najlepszyWynik :: [Wynik] -> Maybe Wynik
-- przykład: najlepszyWynik [("Ala",10),("Jan",17)] == Just ("Jan",17)
```

Zadanie 9. Wersja rozszerzona dla osób, które skończą wcześniej

Proszę zrobić funkcję `tylkoZaliczone`, która zostawia wyłącznie te wyniki, dla których liczba punktów jest nie mniejsza niż podany próg.

```
tylkoZaliczone :: Int -> [Wynik] -> [Wynik]
-- przykład: tylkoZaliczone 10 [("Ala",10),("Jan",7),("Ola",15)]
-- wynik: [("Ala",10),("Ola",15)]
```